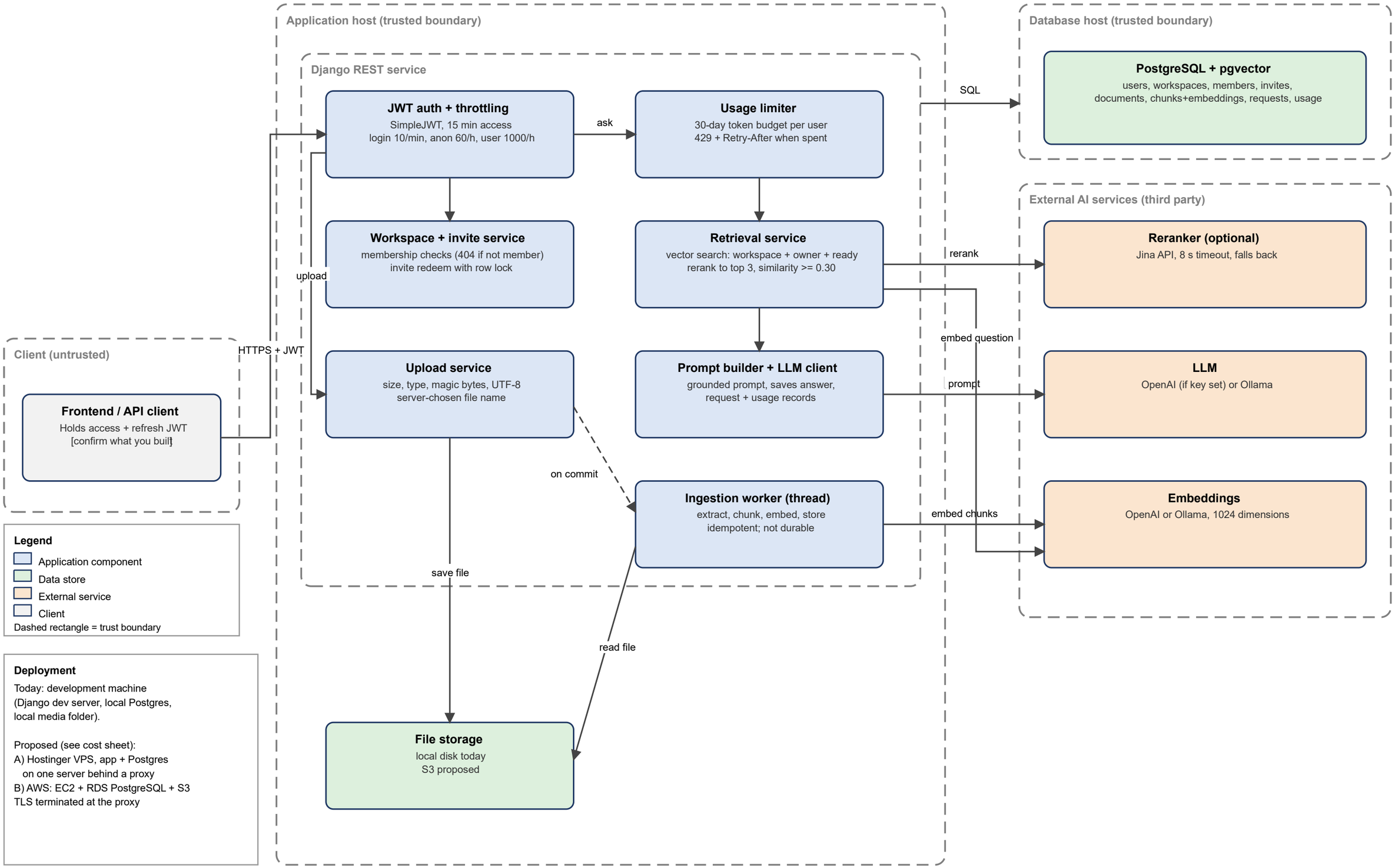
02 - Architecture: AI SaaS Workspace

System architecture showing components, data flows, trust and deployment boundaries. Editable Lucidchart URL: **Not Made**. Drawn from the Django code in this project; dashed rectangles are trust boundaries.



Components and responsibilities

Component	Responsibility	Status
Client	Sends requests, keeps access and refresh tokens	Implemented
JWT auth + throttling	Verifies the Bearer token on every route except register, login, refresh; rate-limits login (10/min), anonymous (60/h), users (1000/h)	Implemented
Workspace + invite service	Creates workspaces; owner-only invites with expiry and use limit; redeeming locks the invite row so a use is not double-counted; non-members get 404	Implemented
Upload service	Checks size (max 20 MB), content type, PDF magic bytes, UTF-8; saves under a server-generated name; creates a pending document and schedules ingestion after commit	Implemented
Ingestion worker	Reads the file, extracts text and tables, chunks (800 chars, 100 overlap), embeds in batches of 16, replaces chunks in one transaction, sets ready or failed	Implemented as a thread
Retrieval service	Embeds the question; SQL vector search limited to the asker's ready documents in the workspace; reranks to top 3	Implemented
Usage limiter	Sums the user's tokens over 30 days; blocks with 429 when the budget is spent	Implemented
Prompt builder + LLM client	Builds the context-only prompt, calls the model, stores answer, token counts and a usage event	Implemented
PostgreSQL + pgvector	System of record and vector storage	Implemented
File storage	Original uploaded files	Local disk; S3 proposed
Embeddings / LLM / Reranker	OpenAI or self-hosted Ollama for embeddings and answers; Jina optional for reranking	Implemented

Data flows

#	Flow	Data	Protection
1	Client to API	Credentials, JWT, questions, uploaded files	HTTPS (at the proxy), JWT, validation, throttling
2	API to database	All records, vectors and usage	Queries scoped by workspace and user; ORM and parameterised SQL
3	API to file storage	Uploaded files, read back during ingestion	Server-chosen names; never executed
4	Upload to ingestion	Document id, after the database commit	Background thread; idempotent re-run
5	API to embedding provider	Document text chunks and questions	API key from environment; text leaves the system when OpenAI is used
6	API to reranker	Question and top candidate chunks	Optional; falls back if it fails or times out
7	API to LLM	Question and retrieved chunks	Prompt tells the model to answer only from context

Trust and security boundaries

Boundary	What crosses it	Controls
Internet to application host	Everything the client sends is untrusted	JWT, input validation, size and type limits, throttling, 404 for non-members
Application to database	SQL from the service only	Every query filters on workspace and user; foreign keys and unique constraints
Application to third-party AI	Questions and document text (OpenAI/Jina); stays local with Ollama	Secrets in environment variables; fixed prompts; provider failures handled as failed requests
Uploaded content	Files and their text are untrusted data	Parsed, never executed; prompt-injection hardening is a known gap

Design trade-offs

Decision	Gain	Cost
Single Django service	Simple to build, test and explain	Everything scales together
pgvector inside Postgres	One datastore; consistent with relational data	Vector load shares the database; no approximate index yet
Ingestion in a thread	No extra infrastructure	Not durable, no retries (queue proposed)
Private documents per user	Strong isolation between members	No shared team knowledge
OpenAI-or-Ollama switch	Cost and privacy flexibility	Two code paths and different token accounting
Similarity threshold before the LLM	Saves tokens and avoids guessed answers	Fixed 0.30 may reject valid questions